If it’s the first time you see big-O notation, it’s perfectly normal to feel slightly confused. It takes some time and lot of practice to get used to it.

Let’s see a few examples and try to apply what we described above in mathematical terms.

Suppose you have 4 numbers you’d like to sum: `{1, 3, -1.4, 7}`.

How many additions will you need? Let’s see:

$$1 + 3 + (-1.4) + 7$$

Looks like you can do with 3 additions. What about if we have to sum 10 numbers? You can easily verify we need 9. Can we generalize this number of operations for any size of the input? Yes, we can, as it’s easy to prove that we always need `n-1` additions to sum up `n` numbers.

So, we can say that summing up the elements of a list of `n` numbers (asymptotically) requires `O(n)` operations. If we denote with `T(n)` the number of operations needed, we can say that `T(n) = O(n)`.

Let’s consider two slight variations:

1. We sum the first 5 elements of a list twice, and the other elements once.
2. We sum the squares of the elements.

In the first case, if our list is `{1, 3, -1.4, 7, 4, 1, 2}`, we have: